Actividad: Operaciones con matrices y arreglos (Processing / Java)

Docente Alexander Espinoza Garcia

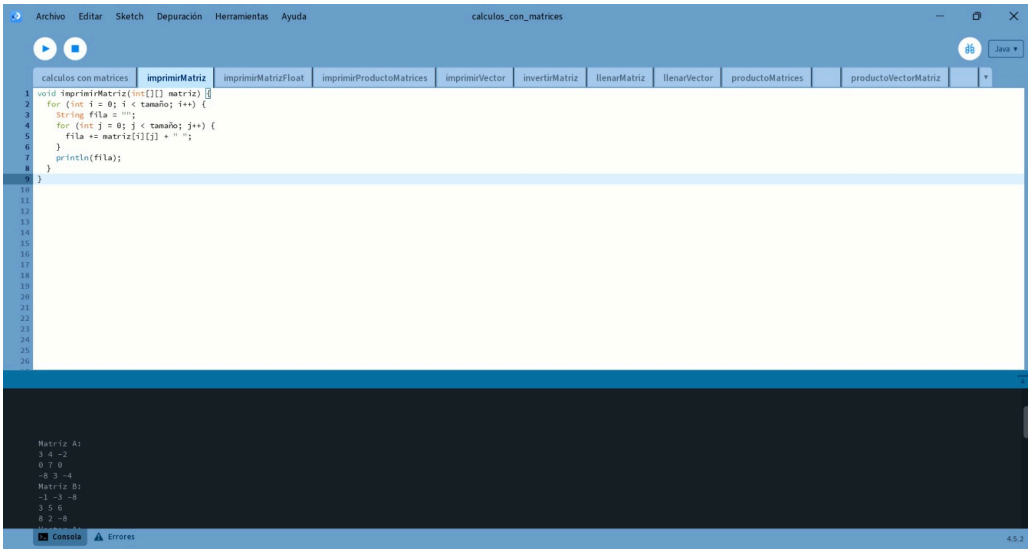
Presentado por:
Lizeth Veronica Chitiva Paez
Cristian Camilo Afanador Acosta

Programa: Ingeniería de Sistemas y Computación
Curso: Programación I

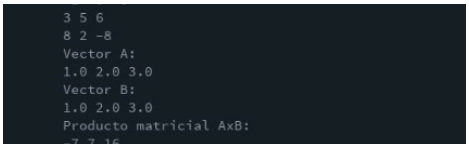
Universidad de Cundinamarca - Extensión Facatativá
9 de Marzo de 2026

Explicación breve del funcionamiento del programa.

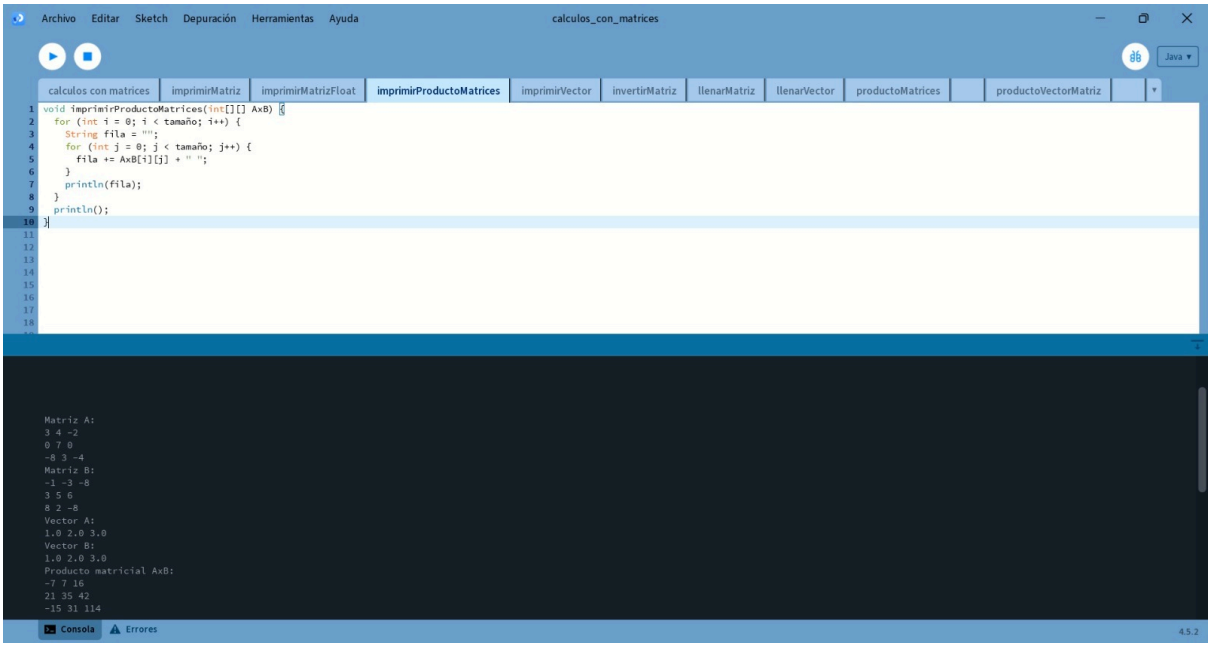
- **Matrices A y B**



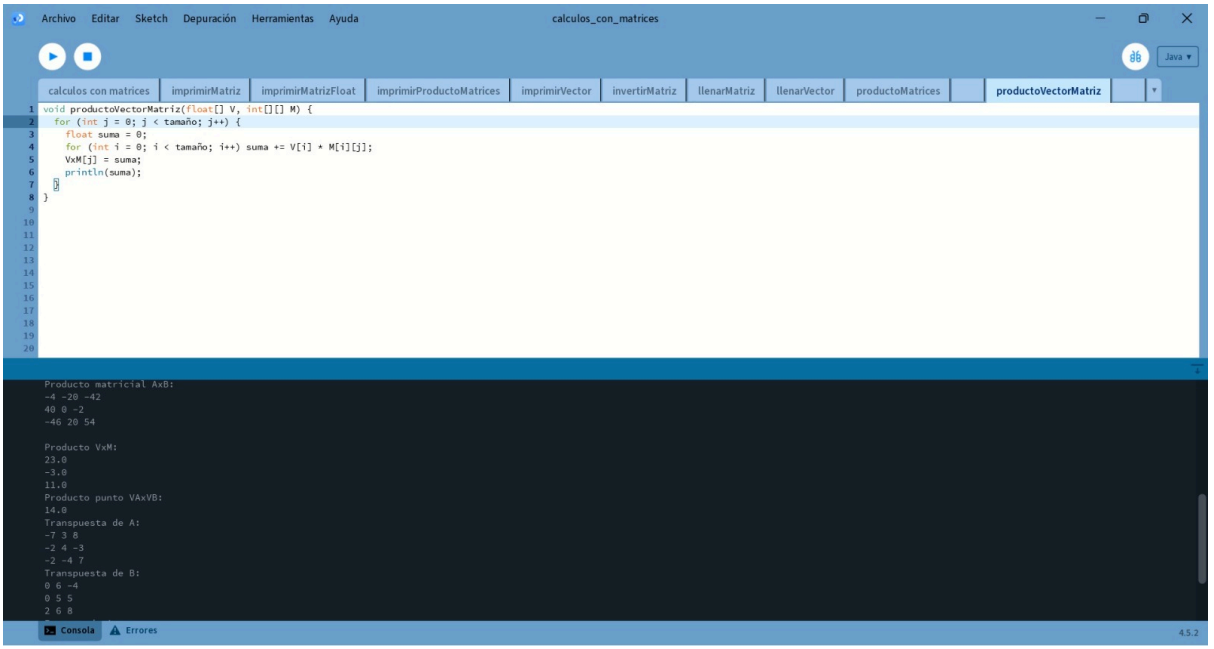
- **Vectores A y B**



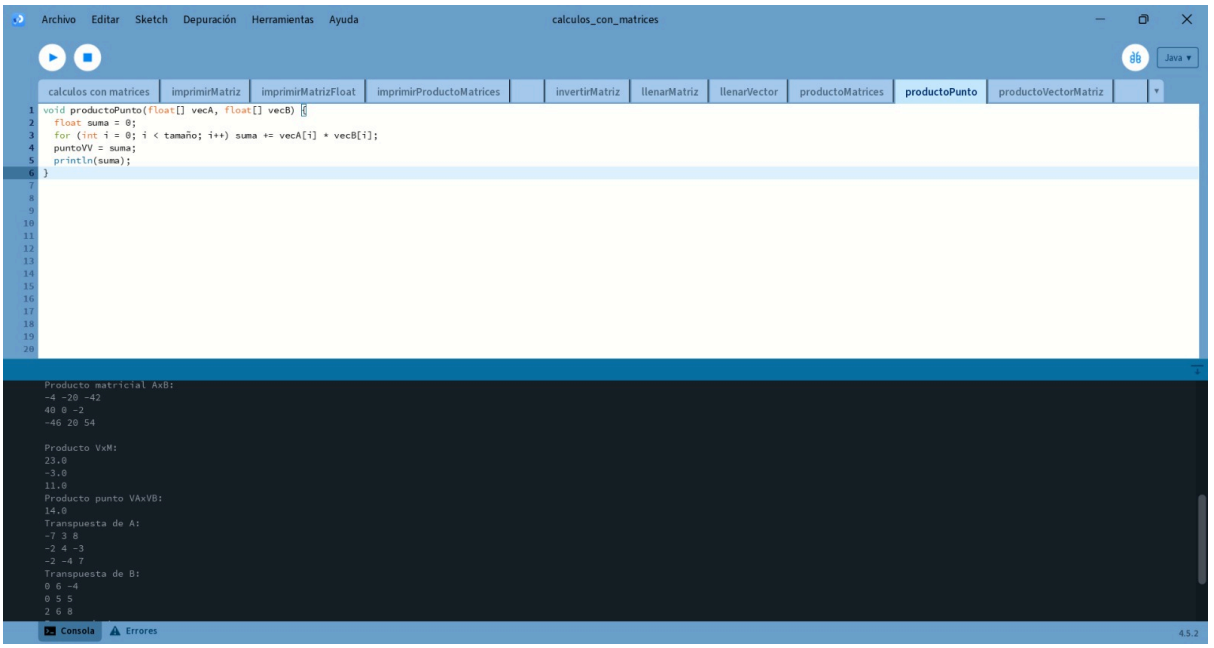
- **Producto de matrices**



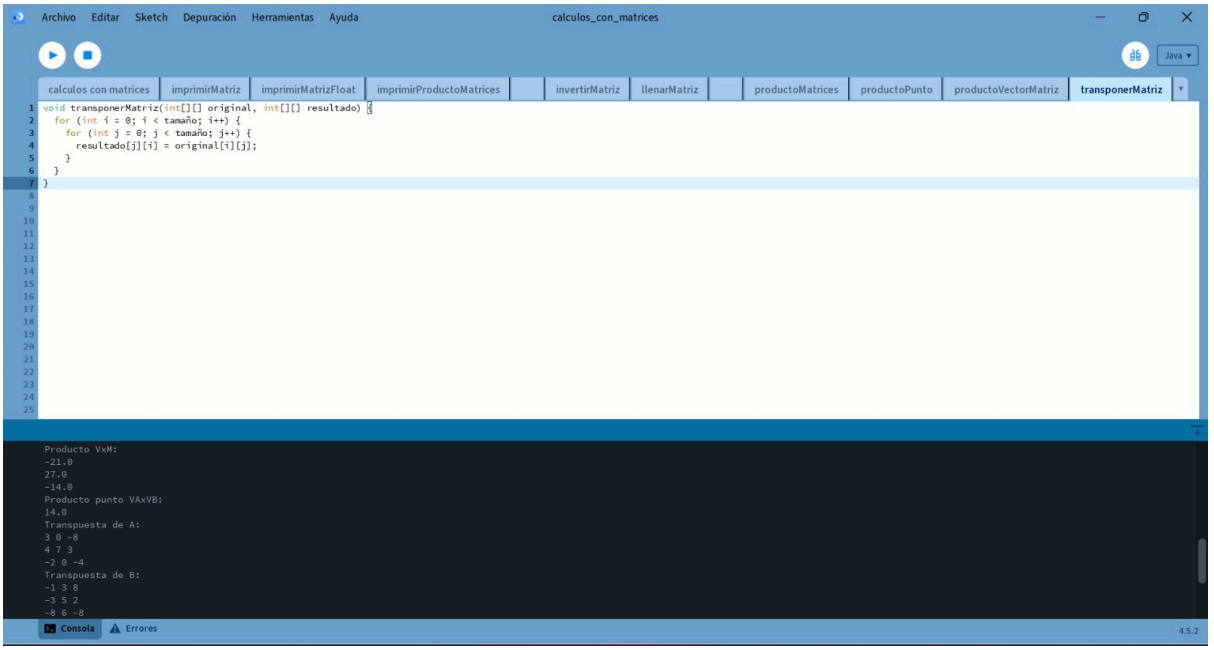
- **Producto vector-matriz**



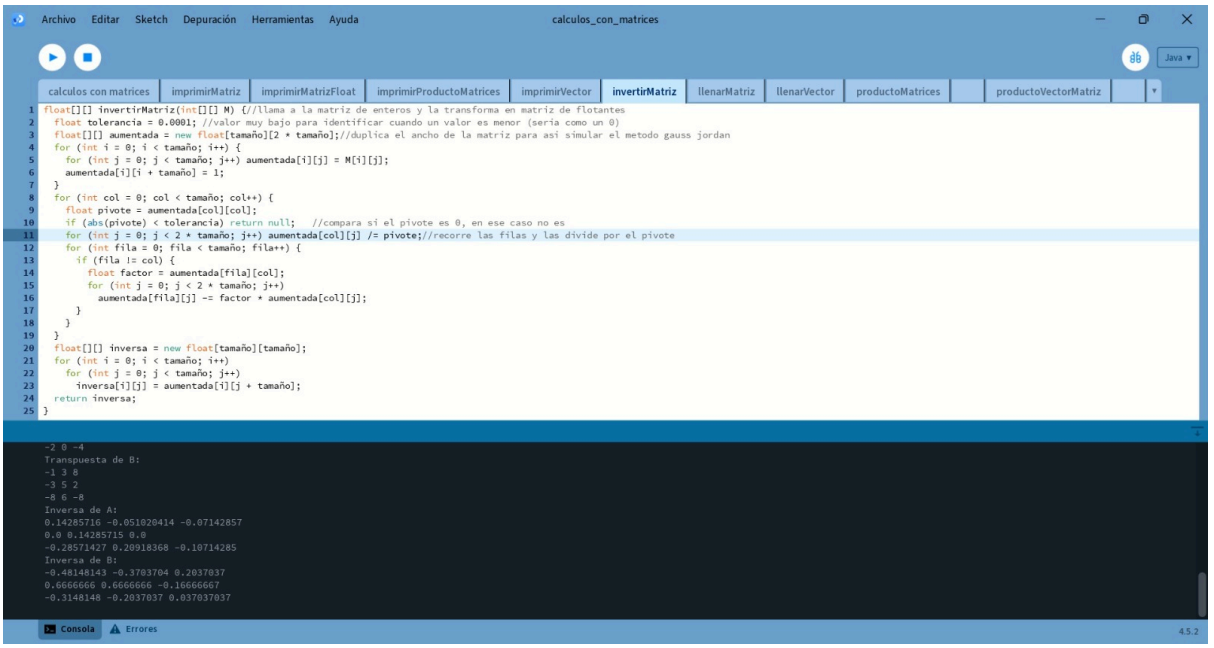
- **Producto punto de arreglos**



- **Matriz transpuesta**



- **Matriz inversa**



★ **Como se llenan las matrices y vectores**

Utilizamos el siguiente código para llenar las matrices de manera aleatoria y de conteo dependiendo lo que se quiera

```
void llenarMatriz(int[][] rellenar, int tipoLlenado) {
  int contador = 1;
  for (int i = 0; i < tamaño; i++) {
    for (int j = 0; j < tamaño; j++) {
      if (tipoLlenado == 1) {
        rellenar[i][j] = int(random(-9, 9));
      } else {
        rellenar[i][j] = contador;
        contador++;
      }
    }
  }
}
```

se solicitan los datos de la matriz por llenar y el tipo de llenado que se necesita, si este es diferente de 1 indica que el llenado es por conteo entonces se inicia una variable de contador para que almacene el dato inicial y de ahi ir llenando fila y columna sumandole de a uno; por otro lado si el tipo de llenado es 1 se llena de manera aleatoria con numeros entre -9 y 9.
Para los vectores se realiza lo mismo pero como estos solo tienen una dimension no es necesario j solo se usa i.

```
void llenarVector(float[] rellenar, int tipoLlenado) {
  int contador = 1;
  for (int i = 0; i < tamaño; i++) {
    if (tipoLlenado == 1) {
      rellenar[i] = int(random(-9, 9));
    } else {
      rellenar[i] = contador;
      contador++;
    }
  }
}
```

★ Como calculamos matriz inversa

```
// funcion para calcular la inversa usando Gauss-Jordan
// retorna null si la matriz no es invertible
float[][] invertirMatriz(int[][] M) {
  float tolerancia = 0.0001;
  float[][] aumentada = new float[tamaño][2 * tamaño];
  for (int i = 0; i < tamaño; i++) {
    for (int j = 0; j < tamaño; j++) aumentada[i][j] = M[i][j];
    aumentada[i][i + tamaño] = 1;
  }
  for (int col = 0; col < tamaño; col++) {
    float pivote = aumentada[col][col];
    if (abs(pivote) < tolerancia) return null;
    for (int j = 0; j < 2 * tamaño; j++) aumentada[col][j] /= pivote;
    for (int fila = 0; fila < tamaño; fila++) {
```

```
      if (fila != col) {
        float factor = aumentada[fila][col];
        for (int j = 0; j < 2 * tamaño; j++)
          aumentada[fila][j] -= factor * aumentada[col][j];
      }
    }
  }
  float[][] inversa = new float[tamaño][tamaño];
  for (int i = 0; i < tamaño; i++)
    for (int j = 0; j < tamaño; j++)
      inversa[i][j] = aumentada[i][j + tamaño];
  return inversa;
}
```

Se tiene que crear primero una matriz que doble el tamaño de la original ya que se debe crear una al lado para luego llenar su diagonal con 1
Recorre las columnas para ver si el pivote es 0 y en ese caso poner null si no continua y divide toda la fila entre el pivote.
Cuando termina todo el ciclo copia las últimas 3 columnas de la matriz aumentada y esa es la inversa.

- ★ Cómo calcular Vector por Matriz

Para esta operación usamos el siguiente código

```
void productoVectorMatriz(float[] V, int[][] M) {
  for (int j = 0; j < tamaño; j++) {
    float suma = 0;
    for (int i = 0; i < tamaño; i++) suma += V[i] * M[i][j];
    VxM[j] = suma;
    println(suma);
  }
}
```

El resultado de esta operación nos da un nuevo vector, lo que nos permite la funcion es multiplicar cada fila de la matriz por el vector y el resultado de la suma de estos 3 nos da el valor en el nuevo vector, y asi hasta obtener los valores de las tres filas para el nuevo vector.